virvelle

CORSO DI MOBILE PROGRAMMING

PROGRAMMA GOL

Virvelle S.r.l. – Società Benefit 2025. All rights reserved. Confidential and proprietary

Finanziato dall'Unione europea
NextGenerationEU

Recap: La Memoria di Ferro

Nella scorsa lezione abbiamo dato alla nostra app una **memoria a lungo termine**:

- **DataStore:** Per le piccole impostazioni (Key-Value).
- **Room Database:** Per mappare tabelle complesse e fare ricerche SQL scrivendo in puro Kotlin.
- Abbiamo scoperto le animazioni e il **Motion UX**.

Oggi facciamo l'ultimo passo: la nostra app prende vita, percepisce il mondo e comunica da sola.

Obiettivi di Oggi

L'App nel Mondo Reale

1. **I Sensi dell'App:** Sensori Hardware e richiami IoT.
2. **Il Muro della Privacy:** Gestione dei Permessi.
3. **L'App che non dorme:** I Servizi (Foreground e Background).
4. **Ascoltare il Sistema:** Broadcast Receivers.
5. **Richiamare l'Attenzione:** Le Notifiche.
6. **Demo Finale:** Costruiamo un "Antifurto" per smartphone.

virvelle

Parte 1: L'App ha dei Sensi

Hardware, Sensori e il mondo IoT

Oltre lo Schermo

Fino ad oggi abbiamo considerato lo smartphone solo come un piccolo schermo con una tastiera.
Ma sotto il vetro c'è un concentrato di **Sensori Elettronici** che lo rendono uno strumento di misurazione scientifica avanzato.

In Android, per parlare con questo hardware, usiamo una classe chiamata `SensorManager`.

Le Tre Famiglie di Sensori

1. 🏃 **Sensori di Movimento:** Accelerometro, Giroscopio, Contapassi. Misurano le forze fisiche applicate al dispositivo.
2. 🌍 **Sensori di Posizione:** GPS, Magnetometro (Bussola). Ci dicono dove siamo nel mondo.
3. 🌡️ **Sensori Ambientali:** Barometro, Sensore di Luce, Termometro (raro oggi). Misurano l'ambiente circostante.

Lo Smartphone come "Edge Device" (IoT)

Voi studiate Elettronica e Telecomunicazioni. Sapete cos'è un Arduino o un ESP32.
Nell'Internet of Things (IoT), i dispositivi che raccolgono dati si chiamano **Edge Devices**.

Oggi, moltissimi progetti industriali usano vecchi smartphone Android come Edge Devices invece di creare hardware personalizzato. Perché?
Ha sensori integrati, Wi-Fi, batteria tampone e uno schermo... per 100€!

Leggere un Sensore (La Logica)

Come fa l'app a sapere se stiamo correndo? Non chiede il dato una volta sola.

Si **iscrive (Listener)** al sensore.

```
// Esempio pseudo-codice della logica del SensorManager
sensorManager.registerListener(
    accelerometro,
    velocitaAggiornamento = "MOLTO_VELOCE"
) { dati ->
    val asseX = dati[0]
    val asseY = dati[1]
    println("Stai muovendo il telefono con forza: $asseX")
}
```

Il lato oscuro dei sensori

L'accelerometro non serve solo a girare lo schermo in orizzontale o a giocare ai giochi di corsa. Un'intelligenza artificiale che analizza i dati dell'accelerometro può capire esattamente:

- Se state camminando, correndo o siete in auto.
- Il vostro **Parkinson** (dai micro-tremori della mano).
- Il vostro livello di ubriachezza.

Dibattito: Assicurazioni e Privacy

Immaginate che un'Assicurazione Auto vi offra uno sconto del 50% sul premio annuale, a patto che installiate la loro app.

L'app usa l'accelerometro e il GPS per capire se frenate bruscamente, se accelerate troppo o se guardate il telefono mentre guidate.

Accettereste questo compromesso? E se diventasse obbligatorio per tutti?

virvelle

Parte 2: Il Muro della Sicurezza

I Permessi (Permissions)

Perché esistono i Permessi?

Nei primi anni di Android (fino al 2015), quando scaricavi un'app, il Play Store ti mostrava un muro di testo: *"Quest'app userà fotocamera, contatti, microfono e GPS"*. L'utente premeva "Accetto" senza leggere e l'app faceva ciò che voleva in background per sempre.

Oggi, Android utilizza un sistema a **Sandboxing** e permessi a Runtime.

La Sandbox (Il recinto di sabbia)

Ogni app su Android è isolata. Vive in un recinto di sabbia chiuso. La mia app "Calcolatrice" non può leggere i file di "WhatsApp".

Per scavalcare il recinto e accedere al mondo (Internet, GPS, Fotocamera), l'app deve chiedere il **Permesso** al Re (Il Sistema Operativo).

Permessi Normali vs Pericolosi

- 🟢 **Permessi Normali:** (Es. Internet, Bluetooth, Allarme). Non ledono la privacy. Li dichiari nel file `AndroidManifest.xml` e il sistema te li accorda automaticamente.
- 🔴 **Permessi Pericolosi:** (Es. Fotocamera, Microfono, Posizione, Rubrica). Sono bloccati di default. L'utente deve premere fisicamente su un **Popup di Sistema** per consentirli.

Dibattito: L'App Torcia

È il 2014. Scaricate dal Play Store una normalissima app "Torcia" per fare luce al buio.

All'avvio, vi compare questo popup: *"Torcia HD vuole accedere ai tuoi Contatti, ai tuoi SMS e alla tua Posizione GPS."*

Perché un'app del genere chiedeva quei permessi? Cosa ci faceva con i vostri dati? (Indizio: i dati sono il nuovo petrolio).

Runtime Permissions (Oggi)

Oggi i permessi si chiedono nel momento esatto in cui servono (Runtime). Se apro WhatsApp, non mi chiede il microfono all'avvio. Me lo chiede **solo quando provo a mandare un vocale per la prima volta**.

Questo dà fiducia all'utente: *"Ah, mi chiede il microfono perché ho appena premuto il tasto del microfono!"*

Il Flusso Logico dei Permessi

Quando scriviamo codice per un permesso pericoloso, non possiamo solo dire "Apri la fotocamera". Dobbiamo gestire 3 casi:

1. **Mai chiesto prima:** Mostriamo il popup.
2. **Accettato:** Apriamo la fotocamera.
3. **Rifiutato:** L'utente ha detto NO. Dobbiamo disabilitare il bottone e spiegare *perché* l'app non può funzionare.

virvelle

I Permessi in Jetpack Compose

In Jetpack Compose, gestire i permessi è diventato molto più pulito grazie alla libreria di Google.

Kotlin

```
// Chiediamo il permesso per la fotocamera
val permessoCamera = rememberPermissionState(
    Permission.CAMERA
)

if (permessoCamera.status.isGranted) {
    Text("Fotocamera Pronta!")
} else {
    Button(onClick = { permessoCamera.launchPermissionRequest() }) {
        Text("Richiedi Accesso")
    }
}
```

UX: Non farsi odiare (La Rationale)

Se l'utente preme "Nega" a un permesso, il sistema operativo (Android 11+) gli permette di bloccare per sempre la richiesta.

Per evitarlo, prima di chiedere il permesso, i bravi sviluppatori mostrano una schermata di **Rationale** (Spiegazione): *"Caro utente, ci serve il GPS per dirti dove hai parcheggiato l'auto. Promettiamo di non inviare questi dati a terzi."*

virvelle

Parte 3: L'App che non dorme

Services e WorkManager

Il limite dell'Activity

Abbiamo imparato che l'Activity (la schermata UI) vive solo finché è visibile (`onResume`). Se premete il tasto "Home", l'app va in `onStop`. Le coroutines si fermano, i download si bloccano, il GPS si spegne.

E se volessimo che un'operazione continuasse in background? Abbiamo bisogno di un **Service** (Servizio).

Cos'è un Service?

Un Service è un componente Android **senza interfaccia grafica**. Gira in background e continua a lavorare anche se l'utente sta usando un'altra app.

Attenzione: Il Service gira di base sul Main Thread! Bisogna comunque usare le Coroutines al suo interno se facciamo operazioni pesanti.

Tipi di Servizi

Ci sono due modi per far sopravvivere il nostro codice "dietro le quinte":

1. **Foreground Service (In Primo Piano)**
2. **Background Service (Dietro le quinte)**

Foreground Service (Farsi Notare)

Android non ama le app che consumano batteria di nascosto. Se volete usare il GPS o suonare musica mentre l'utente usa WhatsApp, dovete usare un **Foreground Service**.

La Regola: Siete obbligati a mostrare una **Notifica ineliminabile**. L'utente deve *sapere* che state consumando le sue risorse (Pensate al navigatore di Google Maps o a Spotify).

Background Service e WorkManager

E se volessimo semplicemente dire all'app: *"Fai il backup del database sul cloud una volta al giorno"*?

Per operazioni differibili nel tempo, non usiamo più i Service tradizionali. Usiamo **WorkManager**.

La Magia del WorkManager

Il WorkManager è il maggiordomo intelligente di Android. Gli date un compito e le **Condizioni** in cui deve svolgerlo.

```
// Esempio logico di WorkManager:
val backupLavoro = PeriodicWorkRequest.Builder(
    BackupDatabaseWorker::class.java,
    24, TimeUnit.HOURS
).setConstraints(
    Constraints.Builder()
        .setRequiresCharging(true) // Solo se in carica!
        .setRequiredNetworkType(NetworkType.UNMETERED) // Solo Wi-Fi!
        .build()
).build()
```

Il sistema sceglierà il momento perfetto per non prosciugare la batteria.

virvelle

IoT Tie-In: MQTT vs HTTP

Come comunicano i veri Edge Devices?

Dal Web all'IoT

Nel Modulo 4 abbiamo usato **HTTP (Retrofit)**. HTTP è perfetto per il web (Apro la pagina -> Ricevo i dati -> Chiudo la connessione).

Ma per l'IoT, l'HTTP è troppo "pesante" e lento. Se ho una serra intelligente, voglio che l'app mi avvisi *istantaneamente* se la temperatura sale troppo, senza dover "ricaricare la pagina".

MQTT: Il protocollo delle Cose

In ambito IoT si usa **MQTT**. È un protocollo leggerissimo basato su **Pub/Sub** (Pubblicazione e Iscrizione).

1. L'ESP32 (Sensore) *pubblica* la temperatura su un "canale" chiamato `serra/temperatura`.
2. La nostra App Android (Service in background) *si iscrive* a quel canale.
3. Appena l'ESP32 invia un dato, l'app lo riceve in un millisecondo.

(Questa è la base della Domotica Moderna: Alexa, Home Assistant).

virvelle

Parte 4: Comunicare con l'Utente

Broadcast e Notifiche

Broadcast Receivers (Le orecchie dell'app)

Android lancia continuamente degli "urli" nel sistema: *"Batteria scesa al 15%!" "Modalità Aereo attivata!" "Schermo acceso!"*

Possiamo registrare un **Broadcast Receiver** nella nostra app per ascoltare questi urli e reagire (es. Salvare i dati al volo se la batteria sta morendo).

Le Notifiche Push

Quando l'app è chiusa o in background, l'unico modo per parlare con l'utente è la **Notifica**.

L'anatomia di una notifica moderna:

- **Small Icon:** L'iconcina nella barra di stato (deve essere bianca su sfondo trasparente).
- **Titolo e Testo.**
- **Azioni Rapide:** Pulsanti direttamente nella tendina (es. "Rispondi", "Segna come letto").

Notification Channels (L'Argine)

Da Android 8.0, Google ha introdotto i **Canali di Notifica**. Siete obbligati a classificare le vostre notifiche.

Esempio in un'app di E-commerce:

- Canale 1: **"Spedizioni"** (Priorità Alta, fa suonare il telefono).
- Canale 2: **"Sconti e Promo"** (Priorità Bassa, silenziosa).

Così l'utente può disattivare le notifiche di marketing *senza* perdersi l'avviso che il suo pacco è arrivato!

Dibattito: L'Economia dell'Attenzione

Molte app (Social Network, Giochi Free-to-play) inviano notifiche finte come: *"A Marco manca la tua fattoria, torna a giocare!"*

Dal punto di vista di uno sviluppatore e di un utente: le notifiche push sono uno strumento di comunicazione vitale o una tecnica di manipolazione psicologica per tenerci incollati allo schermo? Quando è giusto inviare una notifica?

Mettiamoci alla prova!

Kahoot Time 🕹️

L'ultimo quiz in presenza! Vediamo chi si ricorda come funzionano il GPS, i Permessi e il WorkManager!

Kahoot Time!

PIN: 696660

virvelle

Parte 5: Verso la Pratica

L'Antifurto e i Laboratori

Il Progetto di Oggi: L'Antifurto

Uniremo tutto ciò che abbiamo visto oggi in una mini-app **Antifurto**.

1. **Sensore:** Leggeremo l'accelerometro. Se l'app rileva un movimento improvviso...
2. **Servizio:** Faremo partire un suono di allarme a volume massimo che non si spegne chiudendo l'app (Foreground Service).
3. **Notifica:** Mostreremo una notifica con il tasto "Disattiva Allarme".

Roadmap Laboratori a Distanza

Dalla prossima lezione non ci vedremo in aula, ma a distanza. Saranno **7 Laboratori Pratici**. La formula sarà:

1. Sviluppo Guidato (Faccio io, voi copiate e capite l'architettura).
2. Sfida Autonoma (Aggiungete una feature aiutati da me e dall'IA).

Breve Demo 💻

(Condivido lo schermo: Mettiamo l'emulatore sul tavolo virtuale, lanciamo l'Antifurto e muoviamo il telefono... attenti alle orecchie!)

virvelle

virvelle

Learn Your Way, Grow With Us.

Virvelle S.r.l. – Società Benefit 2025. All rights reserved. Confidential and proprietary